



Faculty of Engineering and Technology
Electrical and Computer Engineering Department
DATA BASE SYSTEMS
COMP333
Final PHASE

I-Tech Electronics Store Management System

Prepared by:

Anis DarHammouda #1230834

Aseel Hamed # 1231891

Supervised by:

Dr. Bassem Sayrafi

Section: 2.

Date: 2026/6/18

Table of Contents

Table of Contents	1
Project Scope and Description	3
1. System Overview	3
2. Core System Functions	3
3. Storage and Organization	3
4. Warranty Management	3
5. Technology and Objectives	3
System Requirements	3
1. Functional Requirements	3
2. Non-Functional Requirements	4
Information about the Client	4
1. Client Overview	4
2. Contact Information	4
3. Client Needs	4
Entities:	5
Relationships:	9
Database Design Analysis:	10
1.Total Participation	10
2. Recursive Relationship	11
3.Non-Binary Relationships	12
4.Class Hierarchy	12
ER Diagram	13
ER Diagram Explanation	14
1. General Explanation	14
2. Important Design Decisions	14
3. Relationships Explanation	14
Relational Schema	15
Normalization	17
1. First Normal Form (1NF)	17

2. Second Normal Form (2NF)	17
3. Third Normal Form (3NF)	18
4. Functional Dependencies	19
5. 3NF Justification	21
Advanced Queries	23
Appendix A Advanced SQL Queries	27

Project Scope and Description

1. System Overview

This project is about developing a database application for I-Tech, which is a local electronics store. The store sells different types of products such as smartphones, laptops, and accessories. The main goal of the system is to help organize and manage the daily operations of the store in a clear and efficient way.

2. Core System Functions

The system mainly focuses on organizing product information such as product details, categories, and brands. In addition, it helps track suppliers and customers, while also supporting both sales and purchase transactions.

3. Storage and Organization

The store contains different storage areas, such as a showroom at the front and a storage room, along with other spaces. These areas are divided into multiple storage sections. Each section represents a physical place where products are stored and organized to make access easier.

4. Warranty Management

The system also supports product warranties. Some products, especially devices like smartphones and laptops, may have a warranty, while others, such as some accessories, may not. Warranty information is stored separately and linked only to the products that have a warranty.

5. Technology and Objectives

The system will be developed using Java for the application and MySQL for the database. The main objective is to reduce errors, organize data properly, and make the store's daily work easier and more efficient..

System Requirements

1. Functional Requirements

The system is expected to support a set of main features like:

- Add, update, delete, and view products
- Manage product categories, suppliers, and customers
- Record purchase and sales transactions
- Track inventory quantities
- Generate reports related to stock, sales, and purchases

2. Non-Functional Requirements

The system should meet the following requirements: easy to use, accurate and reliable, connect properly to MySQL, also should support a graphical user interface using JavaFX, and handle daily store operations efficiently

Information about the Client

1. Client Overview

The client is a local electronics store called I-Tech, located in one of the towns in Ramallah. The store sells a variety of electronic products such as smartphones, laptops, chargers, headphones, keyboards, mice, and other accessories.

2. Contact Information

The store can be contacted through the following:

- Phone: (+970-567200596)
- Email: itech@gmail.com

3. Client Needs

The client needs a system that helps organize product data, manage supplier and customer information, track stock levels, and record both sales and purchase transactions. The system should also provide useful reports that help in managing the business more effectively.

NOTE: The entities, attributes, and relationships included in this phase represent an initial design of the system. These elements are not final and may be modified or improved later based on the project requirements and implementation needs.

Entities:

The system is designed based on several entities that represent the main components of the electronics store. These entities were identified according to the system requirements and the operations performed within the store.

Product Entity:

Attribute	Description
Product_ID (PK)	Unique identifier for each product
Category_ID (FK)	Refers to the category of the product
Brand_ID (FK)	Refers to the brand of the product
Product_Name	Name of the product
Model	Product model
Price	Selling price

This entity represents all products available in the store.

Category Entity:

Attribute	Description
Category_ID (PK)	Unique identifier
Category_Name	Name of the category
Unit_Size	Storage space required for one unit of products in this category

Used to classify products.

Supplier Entity:

Attribute	Description
Supplier_ID (PK)	Unique identifier for the supplier
Supplier_Name	Name of the supplier
Company_Name	Company name that the supplier work with
Phone	Phone for contact
Address	Address of the supplier

Stores supplier info.

Customer Entity:

Attribute	Description
Customer_ID (PK)	Unique identifier for customers
Customer_Name	Name of the customer
Phone	Phone for contact

Stores customer info, Sales for unregistered customers are linked to a predefined Walk-in Customer record.

Storage_Section Entity:

Attribute	Description
Section_ID (PK)	Unique identifier for the section
Section_Name	Name of storage section
Section_Type	Type (like showroom or Backroom)
Capacity	Maximum storage capacity of the section
Used_Percentage()	Derived attribute showing the percentage of used capacity

Represents the different storage areas in the store, such as the showroom, backroom, and other sections where products are stored and organized. Each section has a storage capacity, and the used percentage is derived from the stored quantity and capacity.

Purchase Entity:

Attribute	Description
Purchase_ID (PK)	Unique identifier for purchase transaction
Employee_ID (FK)	Foreign key from Employee entity
Supplier_ID (FK)	Foreign key from supplier entity
Purchase_Date	Date of purchase transaction
Delivery_Date	The delivery date of it
Total_Amount	Total cost of the purchase transaction
Purchase_Status	Status of the purchase (Pending, Received, Cancelled)

Represents purchase transactions, the suppliers they are made from, and the employee who records the purchase.

Sale Entity:

Attribute	Description
Sale_ID (PK)	Unique identifier for sale transaction
Employee_ID (FK)	Foreign key from Employee entity
Customer_ID (FK)	Foreign key from Customer entity
Sale_Date	Date
Total_Amount	Total

Represents sale transactions, the customer they are made for, and the employee who records the sale.

Employee Entity:

Attribute	Description
Employee_ID (PK)	Unique identifier for the Employee
Employee_Name	Name of Employee
Position	Working position of Employee
Salary	Employee salary
Phone	Phone number of Employee

Stores employees working in the store.

Brand Entity:

Attribute	Description
Brand_ID (PK)	Unique identifier for Brand
Brand_Name	Name of the Brand
Website	The website of the Brand

Represents product brands, such as Apple, Samsung, and other manufacturers.

Sale_Details Entity:

Attribute	Description
Sale_ID (FK)	Foreign key from Sale entity
Product_ID (FK)	Foreign key from Product entity
Quantity	Quantity of product in the sale transaction
Unit_Price	The price of unit
Discount	Discount applied to the item (optional)

Represents the items included in each sale, including the product, quantity, unit price, and discount.

Purchase_Details Entity:

Attribute	Description
Purchase_ID (FK)	Foreign key from Purchase entity
Product_ID (FK)	Foreign key from Product entity
Quantity	Quantity of product in the Purchase transaction
Unit_Price	The price of unit

Represents the products included in each purchase, along with their quantities and unit prices. Each record corresponds to a specific product within a specific purchase.

Warranty (Weak Entity):

Attribute	Description
Product_ID (FK)	Part of the owner key from Sale_Details
Sale_ID (FK)	Part of the owner key from Sale_Details
Warranty_Number	Partial key for the warranty
Warranty_Type	Type of warranty
Warranty_Period	Period time of warranty
Start_Date	Start date of warranty

Represents warranty information for sold products that include a warranty. Each warranty is associated with a specific sold product in Sale_Details, because the warranty belongs to the sold item, not to the general product. Therefore, Sale_ID and Product_ID reference Sale_Details.

Storage_Details Entity:

Attribute	Description
Product_ID (FK)	Foreign key from Product entity
Section_ID (FK)	Foreign key from Storage_section entity
Quantity	Quantity of product stored

Represents the current stock quantity of the product.

Supplier_Product Entity:

Attribute	Description
Supplier_ID (FK)	Foreign key from Supplier entity
Product_ID (FK)	Foreign key from Product entity
Supply_Price	Price of product

Represents the products supplied by each supplier along with their supply prices.

Payment Entity:

Attribute	Description
Payment_ID (PK)	Unique identifier of payment transaction
Sale_ID (FK)	Foreign key from Sale entity
Payment_Date	Date of payment
Amount	Payment amount
Payment_Method	Method of payment (like cash or installment)
Payment_Status	Payment status (paid or pending)

Represents payments made by customers for sales transactions, including details such as payment method and status. Each payment is associated with a specific sale.

Return_Record Entity:

Attribute	Description
Return_ID (PK)	Unique identifier for each return record
Sale_ID (FK)	Part of a composite foreign key referencing Sale_Details
Product_ID (FK)	Part of a composite foreign key referencing Sale_Details
Return_Date	Date of return
Quantity	Returned quantity
Return_Reason	Reason for return
Return_Status	Status of the return (like: approved, pending)

Represents products that are returned by customers after a sale, including details such as quantity, reason, and status. Each return is associated with a specific product within a specific sale.

Stock_Movement_History Entity:

Attribute	Description
Movement_ID (PK)	Unique identifier for each stock movement
Source_Section_ID (FK)	The section where the product moved from
Destination_Section_ID (FK)	The section where the product moved to
Product_ID (FK)	Foreign key from Product entity
Movement_Type	Type of movement (like: purchase, sale, return, transfer)
Movement_Date	Date of movement
Quantity	Quantity of product moved

Stores records of product movements, including movement type, quantity, date, source section, and destination section. Some section references may be NULL depending on the movement type..

Relationships:

Relationships Overview

Entities	Relationship	Description	Type
Category & Product	belongs to	Each category can include many products and each product belongs to only one category	1:M
Supplier & Supplier_Product	supplies	A supplier can be associated with many supplier-product records and each supplier-product record belongs to one supplier	1:M
Product & Supplier_Product	is supplied by	A product can be associated with many supplier-product records and each supplier-product record belongs to one product	1:M
Customer & Sale	makes	A customer can make many sales, while unregistered sales are linked to a Walk-in Customer record.	1:M
Sale & Sale_Details	includes	A sale can include multiple detail records, and each record belongs to one sale.	1:M
Product & Sale_Details	appears in	A product can appear in multiple sale detail records and each sale detail record refers to one product.	1:M
Purchase & Purchase_Details	includes	Each purchase can include multiple purchase detail records and each purchase detail record belongs to one purchase.	1:M
Product & Purchase_Details	appears in	A product can appear in multiple purchase detail records and each purchase detail record refers to only one product.	1:M
Supplier & Purchase	provides	A supplier can be associated with many purchase transactions and each purchase is linked to one supplier.	1:M
Employee & Employee	supervises	An employee can supervise multiple employees, while an employee may or may not have a supervisor (Recursive Relationship)	1:M
Brand & Product	produces	Each brand can produce many products and each product belongs to one brand.	1:M
Sale_Details & Warranty	has	Each sold product item can have zero or one warranty, and each warranty belongs to one sale detail record.	1:1
Storage_Section & Storage_Details	contains	Each storage section can contain multiple storage detail records and each storage detail record belongs to one storage section.	1:M
Product & Storage_Details	is stored in	A product can appear in multiple storage detail records and each storage detail record refers to one product.	1:M

Relationships Overview (Continued)

Entities	Relationship	Description	Type
Sale & Payment	has	Each sale can have one or more payments and each payment belongs to only one sale.	1:M
Sale_Details & Return_Record	has	Each sale detail record can have zero or more return records and each return record belongs to one sale detail record.	1:M
Product & Stock_Movement_History	has	A product can have multiple stock movement records and each stock movement record belongs to one product.	1:M
Storage_Section & Stock_Movement_History	source_of	A storage section can be the source of many stock movements.	1:M
Storage_Section & Stock_Movement_History	destination_of	A storage section can be the destination of many stock movements.	1:M
Employee & Sale	records	One employee can record many sales, and each sale is recorded by one employee.	1:M
Employee & Purchase	records	One employee can record many purchases, and each purchase is recorded by one employee.	1:M

Database Design Analysis:

1.Total Participation

1) **sale detail & Warranty**

Total participation from Warranty side: every warranty must be linked to a sale detail record.

2) **Customer & Sale**

Total participation from Sale side: Each sale is linked to either a registered customer or the default Walk-in Customer record.

3) **Supplier & Purchase**

Total participation from Purchase side: Every purchase must be linked to a supplier

4) **Sale & Payment**

Total participation from Payment side: Every payment must belong to a sale

5) **Sale_Details & Return_Record**

Total participation from Return_Record side: Every return must be associated with a sale detail record

6) **Product & Stock_Movement_History**

Total participation from Stock_Movement_History side: Every stock movement must be related to a product

7) **Storage_Section & Stock_Movement_History**

Total participation from Stock_Movement_History side: Every stock movement must be associated with at least one storage section as a source section, a destination section, or both depending on the movement type.

8) **Sale & Sale_Details**

Total participation from Sale_Details side: Every sale detail record must belong to a sale

9) **Product & Sale_Details**

Total participation from Sale_Details side: Every sale detail record must refer to a product

10) **Purchase & Purchase_Details**

Total participation from Purchase_Details side: Every purchase detail record must belong to a purchase

11) **Product & Purchase_Details**

Total participation from Purchase_Details side: Every purchase detail record must refer to a product

12) **Supplier & Product_Supplier**

Total participation from Product_Supplier side: Every supplier-product record must be associated with a supplier

13) **Product & Product_Supplier**

Total participation from Product_Supplier side: Every supplier-product record must be associated with a product

14) **Storage_Section & Storage_Details**

Total participation from Storage_Details side: Every storage detail record must belong to a storage section

15) **Product & Storage_Details**

Total participation from Storage_Details side: Every storage detail record must refer to a product

16) **Category & Product**

Total participation from Product side: Every product must belong to a category

17) **Employee & Purchase**

Total participation from Purchase side: Every purchase must be recorded by an employee.

18) **Employee & Sale**

Total participation from Sale side: Every sale must be recorded by an employee.

Note: All other relationships in the diagram have partial participation.

2. Recursive Relationship

The system includes a recursive relationship in: **Employee & Employee**. Since an employee can supervise multiple employees. So each employee may or may not have a supervisor. The type of the relation is One-to-Many (recursive) with partial participation.

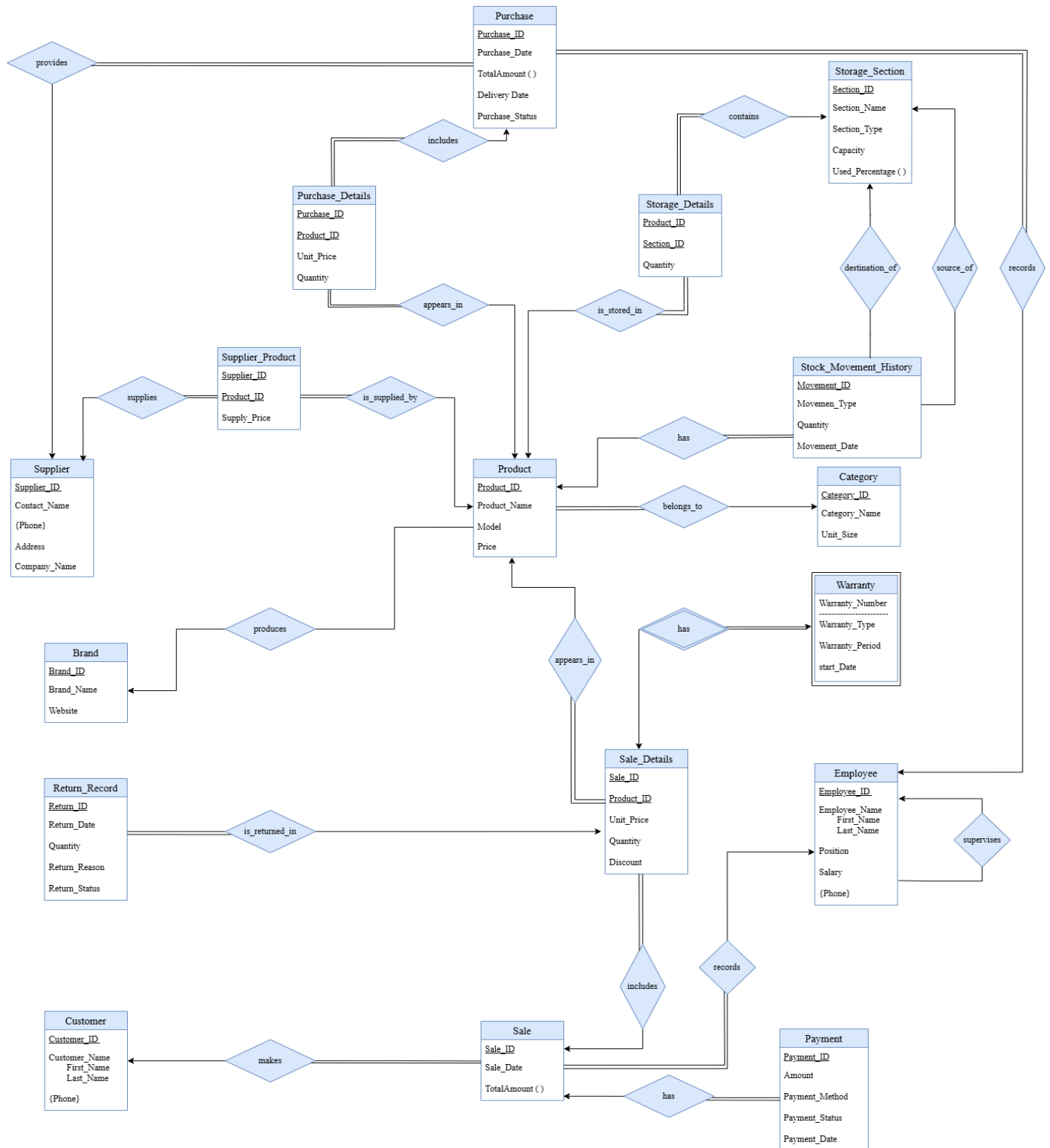
3.Non-Binary Relationships

The current system does not require ternary relationships because all requirements can be represented using binary relationships.

4.Class Hierarchy

Although class hierarchies were considered, the system uses a Category entity to classify products, which provides a simpler and more suitable design for this project.

ER Diagram



ER Diagram Explanation

1. General Explanation

The ER diagram shows the general structure of the database for the electronics store. It explains how the main entities like Product, Sale, Purchase, Customer, and Supplier are connected together using different relationships.

Also, it helps to understand how the data moves inside the system, like in sales, purchases, managing stock, and returns. So basically it gives a full picture of how everything is related.

2. Important Design Decisions

While working on the ER diagram, we made some important decisions to make the design clearer and more organized.

First, the many-to-many relationships (like between Sale and Product, Purchase and Product, Supplier and Product, and Storage_Section and Product) were converted into separate tables. These tables are Sale_Details, Purchase_Details, Product_Supplier, and Storage_Details. This was needed because we also had extra attributes like quantity and price.

Another thing we did was linking Return_Record with Sale_Details instead of linking it directly with Sale and Product. This makes more sense because each return actually belongs to a specific product inside a specific sale, not just the sale in general.

Also, Warranty was designed as a weak entity that depends on Sale_Details and cannot exist alone. This is because a warranty belongs to a specific sold item rather than the product itself. Therefore, Warranty is identified through its relationship with Sale_Details.

In addition, some attributes were modeled differently based on their nature. For example, phone numbers were considered as multi-valued attributes, since a supplier or customer can have more than one phone number. Also, some values such as total amounts can be derived from other attributes, so they were treated as derived attributes..

Finally, we used total participation in some cases where it is required that an entity must be related to another one. For example, every Sale must be linked to a Customer record. If the buyer is not registered in the system, the sale is linked to a predefined Walk-in Customer record.

3. Relationships Explanation

The relationships in the diagram show how the entities interact with each other.

For example, a Customer can make more than one Sale, and each Sale can include multiple products using Sale_Details. Products are also connected to Category and Brand.

Also, stock movement is handled using Stock_Movement_History, which is connected to both Product and Storage_Section to track changes in inventory.

In addition, Employee is linked to both Sale and Purchase entities to record the employee responsible for each transaction. Stock movements are also tracked using source and destination storage sections to provide better inventory control and movement tracking.

Relational Schema

Category(Category_ID PK, Category_Name, Unit_Size)

Brand(Brand_ID PK, Brand_Name, Website)

Product(Product_ID PK, Product_Name, Model, Price, Category_ID FK, Brand_ID FK)

Customer(Customer_ID PK, First_Name, Last_Name)

Customer_Phone(Customer_ID FK, Phone, PK(Customer_ID, Phone))

Supplier(Supplier_ID PK, Contact_Name, Company_Name, Address)

Supplier_Phone(Supplier_ID FK, Phone, PK(Supplier_ID, Phone))

Employee(Employee_ID PK, First_Name, Last_Name, Position, Salary, Supervisor_ID FK)

Employee_Phone(Employee_ID FK, Phone, PK(Employee_ID, Phone))

Sale(Sale_ID PK, Employee_ID FK, Customer_ID FK, Sale_Date)

Sale_Details(Sale_ID FK, Product_ID FK, Quantity, Unit_Price, Discount, PK(Sale_ID, Product_ID))

Purchase(Purchase_ID PK, Employee_ID FK, Supplier_ID FK, Purchase_Date, Delivery_Date, Purchase_Status))

Purchase_Details(Purchase_ID FK, Product_ID FK, Quantity, Unit_Price, PK(Purchase_ID, Product_ID))

Supplier_Product(Supplier_ID FK, Product_ID FK, Supply_Price, PK(Supplier_ID, Product_ID))

Payment(Payment_ID PK, Sale_ID FK, Payment_Date, Amount, Payment_Method, Payment_Status)

Warranty(Sale_ID FK, Product_ID FK, Warranty_Number, Warranty_Type, Warranty_Period, Start_Date, PK(Sale_ID, Product_ID, Warranty_Number))

Return_Record(Return_ID PK, Sale_ID FK, Product_ID FK, Return_Date, Quantity, Return_Reason, Return_Status)

Storage_Section(Section_ID PK, Section_Name, Section_Type, Capacity)

Storage_Details(Product_ID FK, Section_ID FK, Quantity, PK(Product_ID, Section_ID))

Stock_Movement_History(Movement_ID PK, Product_ID FK, Source_Section_ID FK, Destination_Section_ID FK, Movement_Type, Movement_Date, Quantity)

NOTE : Phone numbers were modeled as multi-valued attributes in the ER diagram. During relational mapping, they were converted into separate relations: Customer_Phone, Supplier_Phone, and Employee_Phone.

Design Changes during the Final Phase

During the Final phase, several modifications were made to improve the database design and requirements.

Employee Integration

In the initial design, the Employee entity was only connected through the recursive supervisor relationship. During development, Employee was additionally linked to both Sale and Purchase entities. This modification allows the system to track which employee performed each sale and which employee recorded each purchase, making employee analysis and reporting possible.

Warranty Relationship Update

Initially, Warranty was associated directly with Product. Then the relationship was modified so that warranties are linked to sold products through sales records. This change ensures that warranty information is only created for products that were actually sold.

Inventory Movement Enhancement

The original inventory design only stored product quantities within storage sections. Later, stock movement tracking was expanded by introducing source and destination storage sections in the Stock_Movement_History entity. This improvement allows the system to track product transfers between sections as well as purchases, sales, and returns.

Storage Capacity Tracking

A Capacity attribute was added to the Storage_Section entity to represent the maximum storage space available in each section. In addition, a Unit_Size attribute was added to the Category entity to estimate the storage space occupied by products belonging to each category.

Discount Support

A Discount attribute was added to Sale_Details.

Walk-in Customer Support

A permanent "Walk-in Customer" record was introduced to handle anonymous sales. This design avoids creating unnecessary customer records while maintaining referential integrity and allowing all sales transactions to remain linked to a valid customer record.

Normalization

Normalization was applied to the final relational schema to reduce redundancy and avoid insertion, deletion, and update anomalies. The database design was checked using functional dependencies and normal forms.

1. First Normal Form (1NF)

1- A relation is in 1NF if all attributes contain atomic values and there are no repeating groups or multi-valued attributes.

2- All relations in this database satisfy 1NF because each field stores only one value. For example, product name, price, quantity, date, and phone number are all stored as single atomic values.

3- Phone numbers were not stored as multi-valued attributes inside Customer, Supplier, or Employee. Instead, they were placed in separate relations:

- Customer_Phone(Customer_ID, Phone)
- Supplier_Phone(Supplier_ID, Phone)
- Employee_Phone(Employee_ID, Phone)

This means that if a customer, supplier, or employee has more than one phone number, each number is stored in its own row instead of putting multiple numbers in one field.

Therefore, the database satisfies 1NF.

2. Second Normal Form (2NF)

1- A relation is in 2NF if it is already in 1NF and every non-key attribute is fully functionally dependent on the whole primary key (not just part of it).

2- Relations with a single-column primary key automatically satisfy 2NF, since there is no composite key that could cause a partial dependency. These include:

Category, Brand, Customer, Supplier, Employee, Product, Sale, Purchase, Payment, Return_Record, Storage_Section, Stock_Movement_History.

3- For relations with a composite primary key, each non-key attribute depends on the full key, not on only part of it:

Sale_Details(Sale_ID, Product_ID, Unit_Price, Quantity, Discount):

PK is (Sale_ID, Product_ID). Quantity, Unit_Price and Discount depend on both Sale_ID and Product_ID together, since they describe a specific product inside a specific sale.

Purchase_Details(Purchase_ID, Product_ID, Unit_Price, Quantity):

PK is (Purchase_ID, Product_ID). Unit_Price and Quantity depend on the whole key, since they describe a specific product inside a specific purchase.

Storage_Details(Product_ID, Section_ID, Quantity):

PK is (Product_ID, Section_ID). Quantity depends on both Product_ID and Section_ID, since it represents the amount of a specific product in a specific storage section.

Supplier_Product(Supplier_ID, Product_ID, Supply_Price):

PK is (Supplier_ID, Product_ID). Supply_Price depends on both columns, since the price is specific to a product supplied by a specific supplier.

Warranty(Sale_ID, Product_ID, Warranty_Number, Warranty_Type, Warranty_Period, Start_Date):

PK is (Sale_ID, Product_ID, Warranty_Number). Warranty_Type, Warranty_Period and Start_Date depend on the full key, since they describe the warranty of a specific product sold in a specific sale.

The phone tables also have composite primary keys, but they have no non-key attributes at all (both columns together form the key):

Customer_Phone(Customer_ID, Phone)

Supplier_Phone(Supplier_ID, Phone)

Employee_Phone(Employee_ID, Phone)

Since there are no non-key attributes, there is nothing that could be partially dependent on part of the key, so 2NF is satisfied automatically.

Therefore, the database satisfies 2NF.

3. Third Normal Form (3NF)

1- A relation is in 3NF if it is already in 2NF and there are no transitive dependencies between non-key attributes.

Equivalently, for every functional dependency $X \twoheadrightarrow A$, at least one of the following conditions must hold:

- X is a super key.
- A is part of a candidate key.
- The dependency is trivial.

2- The database satisfies 3NF because descriptive attributes were separated into their own tables when needed.

For example, Product does not store Category_Name or Brand_Name. Instead, it stores Category_ID and Brand_ID as foreign keys:

Product(Product_ID, Product_Name, Model, Price, Category_ID, Brand_ID)
Category(Category_ID, Category_Name)
Brand(Brand_ID, Brand_Name, Website)

This avoids a transitive dependency that would look like this:

Product_ID --> Category_ID --> Category_Name

Instead, Category_Name is stored only in the Category table and reached through Category_ID.

Phone numbers for customers, suppliers, and employees were also separated into their own relations, to avoid repeating the same phone numbers and keep the design normalized.

Sale and Purchase totals are not stored as columns in the Sale or Purchase tables. They can be calculated from Sale_Details, Purchase_Details, and Payment when needed. This avoids storing a derived value that could become inconsistent if the details change later.

A special case worth explaining is Warranty:

Warranty(Sale_ID, Product_ID, Warranty_Number, Warranty_Type, Warranty_Period, Start_Date)

The primary key of the relation is (Sale_ID, Product_ID, Warranty_Number). The attributes Warranty_Type, Warranty_Period, and Start_Date describe a specific warranty record and therefore depend on the full primary key.

The Warranty relation does not contain any repeating groups, partial dependencies, or transitive dependencies. All non-key attributes depend directly on the primary key and not on other non-key attributes.

Therefore, the Warranty relation satisfies Third Normal Form (3NF).

Therefore, the final database schema satisfies 3NF.

4. Functional Dependencies

1- Category:

Category_ID --> Category_Name, Unit_Size

2- Brand:

Brand_ID --> Brand_Name, Website

3- Product:

Product_ID --> Product_Name, Model, Price, Category_ID, Brand_ID

4- Customer:

Customer_ID --> First_Name, Last_Name

5- Customer_Phone:

(Customer_ID, Phone) --> no non-key attributes

6- Supplier:

Supplier_ID --> Contact_Name, Company_Name, Address

7- Supplier_Phone:

(Supplier_ID, Phone) --> no non-key attributes

8- Employee:

Employee_ID --> First_Name, Last_Name, Position, Salary, Supervisor_ID

9- Employee_Phone:

(Employee_ID, Phone) --> no non-key attributes

10- Sale:

Sale_ID --> Employee_ID, Customer_ID, Sale_Date

11- Sale_Details:

(Sale_ID, Product_ID) --> Unit_Price, Quantity, Discount

12- Purchase:

Purchase_ID --> Employee_ID, Supplier_ID, Purchase_Date, Delivery_Date, Purchase_Status

13- Purchase_Details:

(Purchase_ID, Product_ID) --> Unit_Price, Quantity

14- Supplier_Product:

(Supplier_ID, Product_ID) --> Supply_Price

15- Payment:

Payment_ID --> Sale_ID, Amount, Payment_Method, Payment_Status, Payment_Date

16- Warranty:

(Sale_ID, Product_ID, Warranty_Number) --> Warranty_Type, Warranty_Period, Start_Date

17- Return_Record:

Return_ID --> Sale_ID, Product_ID, Return_Date, Quantity, Return_Reason, Return_Status

18- Storage_Section:

Section_ID --> Section_Name, Section_Type, Capacity

19- Storage_Details:

(Product_ID, Section_ID) --> Quantity

20- Stock_Movement_History:

Movement_ID --> Product_ID, Source_Section_ID, Destination_Section_ID, Movement_Type, Quantity, Movement_Date

5. 3NF Justification

NOTE: To verify 3NF, we examined the functional dependencies of each relation. In all relations, the determinant is a primary key or a composite primary key (super key), and no non-key attribute depends on another non-key attribute. Therefore, no transitive dependencies exist and the schema satisfies 3NF.

- **Category:** Category_ID determines Category_Name and Unit_Size.
- **Brand:** Brand_ID determines Brand_Name and Website.
- **Product:** Product_ID determines Product_Name, Model, Price, Category_ID, and Brand_ID. Category and brand info are not stored inside Product, so there is no transitive dependency.
- **Customer:** Customer_ID determines First_Name and Last_Name. Phone numbers are kept separately in Customer_Phone since phone is multi-valued.
- **Supplier:** Supplier_ID determines Contact_Name, Company_Name, and Address. Supplier phone numbers are kept in Supplier_Phone.
- **Employee:** Employee_ID determines First_Name, Last_Name, Position, Salary, and Supervisor_ID. The supervisor is represented as a foreign key to the same Employee table.
- **Sale:** Sale_ID determines Employee_ID, Customer_ID, and Sale_Date. The sale total is not stored and can be calculated when needed.
- **Sale_Details:** the composite key (Sale_ID, Product_ID) determines Unit_Price, Quantity, and Discount, which all describe that product inside that sale.
- **Purchase:** Purchase_ID determines Employee_ID, Supplier_ID, Purchase_Date, Delivery_Date, and Purchase_Status. The purchase total is not stored and can be calculated from Purchase_Details.
- **Purchase_Details:** the composite key (Purchase_ID, Product_ID) determines Unit_Price and Quantity.
- **Supplier_Product:** the composite key (Supplier_ID, Product_ID) determines Supply_Price.

- **Payment:** Payment_ID determines Sale_ID, Amount, Payment_Method, Payment_Status, and Payment_Date.
- **Warranty:** the composite key (Sale_ID, Product_ID, Warranty_Number) determines Warranty_Type, Warranty_Period, and Start_Date. Therefore, all non-key attributes depend on the full primary key.
- **Return_Record:** Return_ID determines Sale_ID, Product_ID, Return_Date, Quantity, Return_Reason, and Return_Status.
- **Storage_Section:** Section_ID determines Section_Name, Section_Type, and Capacity.
- **Storage_Details:** the composite key (Product_ID, Section_ID) determines Quantity.
- **Stock_Movement_History:** Movement_ID determines Product_ID, Source_Section_ID, Destination_Section_ID, Movement_Type, Quantity, and Movement_Date.
- **Customer_Phone, Supplier_Phone, Employee_Phone:** these relations have no non-key attributes (the two columns together form the key), so 2NF and 3NF are satisfied trivially.

As a result, the final database design avoids repeating groups, partial dependencies, and transitive dependencies. Therefore, the database satisfies 1NF, 2NF, and 3NF.

Advanced Queries

1. Products per Category : Display each category name and the total number of products in that category, sorted by the total number of products in descending order.

SQL Code: Appendix A, Query 1.

2. Average Price per Category: Retrieve each category name and the average selling price of products in that category, sorted by the average price in descending order.

SQL Code: Appendix A, Query 2.

3. Most Expensive Product per Brand: Show the most expensive product for each brand along with its name and price, ordered from highest to lowest price.

SQL Code: Appendix A, Query 3.

4. Customer Sales Details: List customer information together with sale details, purchased product names, quantities, item totals, and total payments, sorted by Customer ID and Sale ID.

SQL Code: Appendix A, Query 4.

5. Search Customer Sales Details: Retrieve customer sales details for customers whose first name or last name matches the search value, including purchased products, quantities, item totals, and total payments, sorted by Customer ID and Sale ID.

SQL Code: Appendix A, Query 5.

6. Employee Work Summary: Show employee details along with their position, manager, sales count, purchase count, and total work records.

SQL Code: Appendix A, Query 6.

7. Employee Work Search: Retrieve employee work summary records where the employee first name or last name matches the search value.

SQL Code: Appendix A, Query 7.

8. Employee Sales Details: Display employee sales records, including employee information, sale ID, customer name, number of products sold, and total sale payment.

SQL Code: Appendix A, Query 8.

9. Search Employee Sales Details: Retrieve employee sales records where the employee first name or last name matches the search value.

SQL Code: Appendix A, Query 9.

10. Employee Purchase Details: Retrieve each employee purchase record with employee information, purchase ID, supplier name, number of products purchased, and total purchase value.

SQL Code: Appendix A, Query 10.

11. Search Employee Purchase Details: Search and display purchase records for employees based on a matching first name or last name.

SQL Code: Appendix A, Query 11.

12. Storage Capacity Alert: Retrieve the number of storage sections that exceed their available capacity by calculating the used storage space in each section and comparing it with the section capacity.

SQL Code: Appendix A, Query 12.

13. Storage Capacity Details: Generate a report showing storage section details, total capacity, used storage space, and utilization percentage.

SQL Code: Appendix A, Query 13.

14. Search Storage Capacity Details: Show storage section details, including used space and capacity information, for sections whose names match the search criteria.

SQL Code: Appendix A, Query 14.

Query 15: Top Customers by Purchases :Retrieve customers ranked according to the total value of products purchased.

SQL Code: Appendix A, Query 15.

Query 16: Most Supplied Product :List products and their total supplied quantities based on purchase records, ranked from highest to lowest.

SQL Code: Appendix A, Query 16.

Query 17: Top Suppliers by Supply Value :Retrieve suppliers ranked according to the total value of products supplied to the company.

SQL Code: Appendix A, Query 17.

Query 18: Top Employees by Sales Count :List employees and their sales transaction counts, ranked from highest to lowest.

SQL Code: Appendix A, Query 18.

Query 19: Salary per Position :Generate a report showing the highest salary for each employee position.

SQL Code: Appendix A, Query 19.

Query 20: Total Inventory Value :Display product information, available stock quantities, product prices, and the calculated inventory value for each product.

SQL Code: Appendix A, Query 20.

Query 21: Empty Storage Sections:Retrieve storage sections that currently contain no products.

SQL Code: Appendix A, Query 21.

Query 22: Top 10 Sales :Show the top sales transactions based on their final values after discount deductions.

SQL Code: Appendix A, Query 22.

Query 23: Customer Sales Summary :Retrieve each customer's information, total sales count, and total sales value within a selected date range.

SQL Code: Appendix A, Query 23.

Query 24: Profit by Period :Show financial performance for a selected date range by calculating total revenue, total cost, and profit.

SQL Code: Appendix A, Query 24.

Query 25: Purchase vs Sale Price Comparison: Compare each product's average supply price and average sale price, and calculate the profit margin based on the difference between purchasing and selling prices.

SQL Code: Appendix A, Query 25.

Query 26: Warranty Expiry Overview: Retrieve warranty records including product information, customer information, warranty type, warranty period, and warranty start date.

SQL Code: Appendix A, Query 26.

Query 27 : Recent Sales: Display the most recent sales transactions along with customer and employee information.

SQL Code: Appendix A, Query 27 .

Query 28 : Top Products by Quantity Sold: Retrieve the most sold products based on total quantity sold across all sales transactions.

SQL Code: Appendix A, Query 28 .

Query 29 : Available Storage Sections by Free Capacity: Retrieve storage sections and calculate used capacity based on product quantities and category unit sizes to determine available space.

SQL Code: Appendix A, Query 29 .

Query 30 Load Purchases with Totals: Retrieve purchase information together with supplier and employee names and calculate purchase totals and item counts.

SQL Code: Appendix A, Query 30 .

Query 31 : Check Section Capacity Before Adding Items: Calculate used storage capacity and determine whether a storage section has enough available space for incoming products.

SQL Code: Appendix A, Query 31 .

Query 32: Revenue Calculation: Calculate net revenue by subtracting the value of approved returned products from total sales revenue.

SQL Code: Appendix A, Query 32 .

Query 33: Monthly Sales, Revenue, and Profit Analysis: Generate monthly statistics by calculating sales count, revenue, and profit using date functions and aggregate operations.

SQL Code: Appendix A, Query 33 .

Query 34: Last Payment per Pending Sale: Retrieve the latest payment information for sales that still have outstanding balances.

SQL Code: Appendix A, Query34 .

Appendix A Advanced SQL Queries

Query 1: Products per Category

```
select c.Category_Name, COUNT(p.Product_ID) as Total
from Category c,Product p
where c.Category_ID = p.Category_ID
group by c.Category_Name
order by Total desc
```

Query 2: Average Price per Category

```
select c.Category_Name, AVG(p.Price) as AvgPrice
from Category c, Product p
where c.Category_ID = p.Category_ID
group by c.Category_Name
order by AvgPrice desc
```

Query 3: Most Expensive Product per Brand:

```
select b.Brand_Name, p.Product_Name, p.Price
from Brand b, Product p
where b.Brand_ID = p.Brand_ID
and p.Price =(select MAX(p2.Price)
from Product p2
where p2.Brand_ID = b.Brand_ID )
order by p.Price desc
```

Query 4: Customer Sales Details

```
select c.Customer_ID,c.First_Name,c.Last_Name,s.Sale_ID,p.Product_Name,sd.Quantity,
((sd.Unit_Price * sd.Quantity) - sd.Discount) as line_total, (select sum(pay.Amount)
from Payment pay
where pay.Sale_ID = s.Sale_ID ) as paid_total
from Customer c,
Sale s,Sale_Details sd,Product p
where c.Customer_ID = s.Customer_ID
and s.Sale_ID = sd.Sale_ID
and sd.Product_ID = p.Product_ID
order by c.Customer_ID,s.Sale_ID
```

Query 5: Search Customer Sales Details

```
select c.Customer_ID, c.First_Name, c.Last_Name, s.Sale_ID, p.Product_Name, sd.Quantity,  
((sd.Unit_Price * sd.Quantity) - sd.Discount) as line_total, (select sum(pay.Amount)  
from Payment pay  
where pay.Sale_ID = s.Sale_ID) as paid_total  
from Customer c, Sale s, Sale_Details sd, Product p  
where c.Customer_ID = s.Customer_ID  
and s.Sale_ID = sd.Sale_ID  
and sd.Product_ID = p.Product_ID  
and (c.First_Name like '%value%' or c.Last_Name like '%value%')  
order by c.Customer_ID, s.Sale_ID
```

Query 6: Employee Work Summary

```
select e.Employee_ID, e.First_Name, e.Last_Name, e.Position,  
s.First_Name as MFirst, s.Last_Name as MLast, (select count(*) from Sale sa where sa.Employee_ID =  
e.Employee_ID) as sales_count, (select count(*) from Purchase p where p.Employee_ID = e.Employee_ID)  
as purchase_count  
from Employee e left join Employee s on e.Supervisor_ID = s.Employee_ID  
order by e.Employee_ID
```

Query 7: Employee Work Search

```
select e.Employee_ID, e.First_Name, e.Last_Name, e.Position,  
s.First_Name as MFirst, s.Last_Name as MLast, (select count(*) from Sale sa where sa.Employee_ID =  
e.Employee_ID) as sales_count, (select count(*) from Purchase p where p.Employee_ID = e.Employee_ID)  
as purchase_count  
from Employee e left join Employee s on e.Supervisor_ID = s.Employee_ID  
where e.First_Name like '% value %' or e.Last_Name like '% value %'
```

Query 8: Employee Sales Details

```
select e.Employee_ID, e.First_Name, e.Last_Name, s.Sale_ID,  
c.First_Name as CFirst, c.Last_Name as CLast, (select count(*) from Sale_Details sd where  
sd.Sale_ID=s.Sale_ID) as products_sold, (select sum(pay.Amount) from Payment pay where  
pay.Sale_ID=s.Sale_ID) as sale_total  
from Employee e, Sale s, Customer c  
where e.Employee_ID=s.Employee_ID  
and c.Customer_ID=s.Customer_ID  
order by s.Sale_ID
```

Query 9: Search Employee Sales Details

```
select e.Employee_ID, e.First_Name, e.Last_Name, s.Sale_ID, c.First_Name as CFirst, c.Last_Name as CLast, (select count(*) from Sale_Details sd where sd.Sale_ID=s.Sale_ID) as products_sold, (select sum(pay.Amount) from Payment pay where pay.Sale_ID=s.Sale_ID) as sale_total
from Employee e, Sale s, Customer c
where e.Employee_ID=s.Employee_ID
and c.Customer_ID=s.Customer_ID
and (e.First_Name like '% value %' or e.Last_Name like '% value %')
```

Query 10: Employee Purchase Details

```
select e.Employee_ID, e.First_Name, e.Last_Name, p.Purchase_ID,
s.Contact_Name, (select count(*) from Purchase_Details pd where pd.Purchase_ID=p.Purchase_ID) as
products_purchased, (select sum(pd.Unit_Price * pd.Quantity) from Purchase_Details pd where
pd.Purchase_ID=p.Purchase_ID) as purchase_total
from Employee e, Purchase p, Supplier s
where e.Employee_ID=p.Employee_ID
and s.Supplier_ID=p.Supplier_ID
order by p.Purchase_ID
```

Query 11: Search Employee Purchase Details

```
select e.Employee_ID, e.First_Name, e.Last_Name, p.Purchase_ID, s.Contact_Name, (select count(*) from
Purchase_Details pd where pd.Purchase_ID=p.Purchase_ID) as products_purchased, (select
sum(pd.Unit_Price * pd.Quantity) from Purchase_Details pd where pd.Purchase_ID=p.Purchase_ID) as
purchase_total
from Employee e, Purchase p, Supplier s
where e.Employee_ID=p.Employee_ID
and s.Supplier_ID=p.Supplier_ID
and (e.First_Name like '% value %' or e.Last_Name like '% value %')
```

Query 12: Storage Capacity Alert

```
select ss.Section_ID, ss.Capacity,
sum(sd.Quantity * c.Unit_Size) as used_q
from Storage_Section ss left join Storage_Details sd on ss.Section_ID = sd.Section_ID left join Product p
on sd.Product_ID = p.Product_ID left join Category c on p.Category_ID = c.Category_ID
group by ss.Section_ID, ss.Capacity
having used_q > ss.Capacity
```

Query 13: Storage Capacity Details

```
select ss.Section_ID, ss.Section_Name, ss.Section_Type,  
ss.Capacity, sum(sd.Quantity * c.Unit_Size) as used_q  
from Storage_Section ss left join Storage_Details sd on ss.Section_ID = sd.Section_ID left join Product p  
on sd.Product_ID = p.Product_ID left join Category c on p.Category_ID = c.Category_ID  
group by ss.Section_ID, ss.Section_Name,  
ss.Section_Type, ss.Capacity  
order by ss.Section_ID
```

Query 14: Search Storage Capacity Details

```
select ss.Section_ID, ss.Section_Name, ss.Section_Type,  
ss.Capacity, sum(sd.Quantity * c.Unit_Size) as used_q  
from Storage_Section ss left join Storage_Details sd on ss.Section_ID = sd.Section_ID left join Product p  
on sd.Product_ID = p.Product_ID left join Category c on p.Category_ID = c.Category_ID  
where ss.Section_Name like '%value%'  
group by ss.Section_ID, ss.Section_Name, ss.Section_Type, ss.Capacity  
order by ss.Section_ID
```

Query 15: Top Customers by Purchases

```
select c.Customer_ID, c.First_Name, c.Last_Name, sum(sd.Unit_Price * sd.Quantity) - sum(sd.Discount)  
as total_bought  
from Customer c, Sale s, Sale_Details sd  
where c.Customer_ID = s.Customer_ID  
and s.Sale_ID = sd.Sale_ID  
and c.Customer_ID <> 1  
group by c.Customer_ID, c.First_Name, c.Last_Name  
order by total_bought desc
```

Query 16: Most Supplied Product

```
select p.Product_ID, p.Product_Name, sum(pd.Quantity) as Total_Supplied  
from Product p, Purchase_Details pd  
where p.Product_ID = pd.Product_ID  
group by p.Product_ID, p.Product_Name  
order by Total_Supplied desc
```

Query 17: Top Suppliers by Supply Value

```
select s.Supplier_ID, s.Contact_Name, s.Company_Name, sum(pd.Unit_Price * pd.Quantity) as
total_supplied
from Supplier s, Purchase p, Purchase_Details pd
where s.Supplier_ID = p.Supplier_ID
and p.Purchase_ID = pd.Purchase_ID
group by s.Supplier_ID, s.Contact_Name, s.Company_Name
order by total_supplied desc
```

Query 18: Top Employees by Sales Count

```
select e.Employee_ID, e.First_Name, e.Last_Name, count(s.Sale_ID) as Sales_Count
from Employee e, Sale s
where e.Employee_ID = s.Employee_ID
group by e.Employee_ID, e.First_Name, e.Last_Name
order by Sales_Count desc
```

Query 19: Salary per Position

```
select Position, max(Salary) as max_salary
from Employee
group by Position
order by max_salary desc
```

Query 20: Total Inventory Value

```
select p.Product_ID, p.Product_Name, sum(sd.Quantity) as Total_Stock, p.Price,(sum(sd.Quantity) *
p.Price) as Inventory_Value
from Product p, Storage_Details sd
where p.Product_ID = sd.Product_ID
group by p.Product_ID, p.Product_Name, p.Price
order by Inventory_Value desc
```

Query 21: Empty Storage Sections

```
select ss.Section_ID, ss.Section_Name, ss.Section_Type, ss.Capacity
from Storage_Section ss left join Storage_Details sd on ss.Section_ID = sd.Section_ID
group by ss.Section_ID, ss.Section_Name, ss.Section_Type, ss.Capacity
having sum(sd.Quantity) = 0 or sum(sd.Quantity) is null
```

Query 22: Top 10 Sales

```
select s.Sale_ID, s.Sale_Date, sum(sd.Unit_Price * sd.Quantity) - sum(sd.Discount) as sale_value
from Sale s, Sale_Details sd
where s.Sale_ID = sd.Sale_ID
group by s.Sale_ID, s.Sale_Date
order by sale_value desc
```

Query 23: Customer Sales Summary

```
select c.Customer_ID, c.First_Name, c.Last_Name, (select count(*) from Sale s2 where s2.Customer_ID =
c.Customer_ID and s2.Sale_Date between 'fromDate' and 'toDate') as Sales_Count, sum(sd.Unit_Price *
sd.Quantity) - sum(sd.Discount) as Total_Sales
from Customer c, Sale s, Sale_Details sd
where c.Customer_ID = s.Customer_ID
and s.Sale_ID = sd.Sale_ID
and s.Sale_Date between 'fromDate' and 'toDate'
and c.Customer_ID <> 1
group by c.Customer_ID,c.First_Name,c.Last_Name
order by Total_Sales desc
```

Query 24: Profit by Period

```
Select sum(sd.Unit_Price * sd.Quantity) - sum(sd.Discount) as revenue, sum(sd.Quantity * (select
avg(sp.Supply_Price)
from Supplier_Product sp
where sp.Product_ID = sd.Product_ID)) as cost
from Sale s, Sale_Details sd
where s.Sale_ID = sd.Sale_ID
and s.Sale_Date between 'fromDate' and 'toDate'
```

Query 25: Purchase vs Sale Price Comparison

```
select p.Product_ID, p.Product_Name, avg(sp.Supply_Price) as avg_supply, avg(sd.Unit_Price) as
avg_sale, avg(sd.Unit_Price) - avg(sp.Supply_Price) as margin
from Product p join Supplier_Product sp on p.Product_ID = sp.Product_ID join Sale_Details sd on
p.Product_ID = sd.Product_ID
group by p.Product_ID, p.Product_Name
order by margin desc
```

Query 26: Warranty Expiry Overview

```
select p.Product_Name,c.First_Name,c.Last_Name,w.Warranty_Type,w.Warranty_Period,w.Start_Date
from Warranty w,Sale_Details sd,Product p,Sale s,Customer c
where w.Sale_ID = sd.Sale_ID and w.Product_ID = sd.Product_ID and sd.Product_ID = p.Product_ID
and w.Sale_ID = s.Sale_ID and s.Customer_ID = c.Customer_ID
order by w.Start_Date desc
```

Query 27: Recent Sales

```
select s.Sale_ID, c.First_Name, c.Last_Name, e.First_Name as EFirst, e.Last_Name as ELast, s.Sale_Date
from Sale s, Customer c, Employee e
where s.Customer_ID = c.Customer_ID
and s.Employee_ID = e.Employee_ID
order by s.Sale_Date desc, s.Sale_ID desc;
```

Query 28: Top Products by Quantity Sold

```
select p.Product_ID, p.Product_Name, sum(sd.Quantity) as SoldQty
from Sale_Details sd, Product p
where sd.Product_ID = p.Product_ID
group by p.Product_ID, p.Product_Name
order by SoldQty desc;
```

Query 29: Available Storage Sections by Free Capacity

```
SELECT ss.Section_ID, ss.Section_Name, ss.Capacity,
SUM(sd.Quantity * c.Unit_Size) AS UsedCapacity
FROM Storage_Section ss
LEFT JOIN Storage_Details sd ON ss.Section_ID = sd.Section_ID
LEFT JOIN Product p ON sd.Product_ID = p.Product_ID
LEFT JOIN Category c ON p.Category_ID = c.Category_ID
GROUP BY ss.Section_ID, ss.Section_Name, ss.Capacity
ORDER BY ss.Section_ID;
```

Query 30: Load Purchases with Totals

```
SELECT p.Purchase_ID, s.Company_Name, e.First_Name, e.Last_Name, p.Purchase_Date,
p.Delivery_Date, p.Purchase_Status
FROM Purchase p, Supplier s, Employee e
WHERE p.Supplier_ID = s.Supplier_ID
AND p.Employee_ID = e.Employee_ID
ORDER BY p.Purchase_ID DESC;
```

```
SELECT SUM(Quantity * Unit_Price) AS Total, SUM(Quantity) AS Items
FROM Purchase_Details
WHERE Purchase_ID = -;
```

Query 31: Check Section Capacity Before Adding Items

```
SELECT ss.Section_ID, ss.Section_Name, ss.Capacity, SUM(sd.Quantity * c.Unit_Size) AS UsedCapacity
FROM Storage_Section ss LEFT JOIN Storage_Details sd ON ss.Section_ID = sd.Section_ID
LEFT JOIN Product p ON sd.Product_ID = p.Product_ID
LEFT JOIN Category c ON p.Category_ID = c.Category_ID
GROUP BY ss.Section_ID, ss.Section_Name, ss.Capacity
ORDER BY ss.Section_ID;
```

Query 32: Revenue Calculation

```
SELECT SUM(Unit_Price * Quantity - Discount)
FROM Sale_Details;
```

```
SELECT SUM(r.Quantity * (sd.Unit_Price - sd.Discount / sd.Quantity))
FROM Return_Record r, Sale_Details sd
WHERE r.Sale_ID = sd.Sale_ID
AND r.Product_ID = sd.Product_ID
AND r.Return_Status = 'Approved';
```

Query 33: Monthly Sales, Revenue, and Profit Analysis

```
SELECT COUNT(*) AS SaleCount
FROM Sale
WHERE MONTH(Sale_Date) = - AND YEAR(Sale_Date) = -;
```

```
SELECT AVG(Supply_Price) AS AvgSupply
FROM Supplier_Product
WHERE Product_ID = -;
```

Query 34: Last Payment per Pending Sale:

```
select Payment_Method, Payment_Date
from Payment
where Sale_ID = saleId
order by Payment_Date desc, Payment_ID desc;
```